# AMRUTA INSTITUTE OF ENGINEERING AND MANAGEMENT SCIENCES – POLYTECHNIC

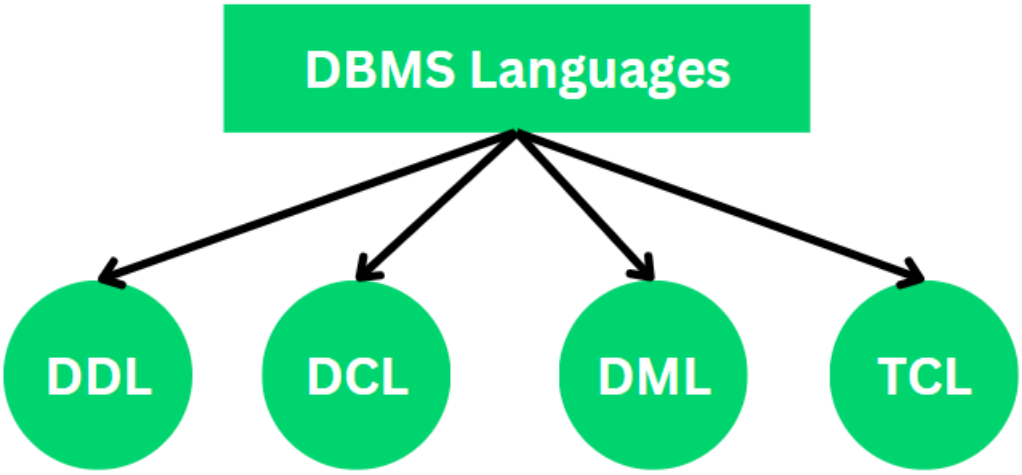
## Course: Database System Concepts and PL/SQL

## Course Code: 20CS34P

## Week 5

## DATABASE LANGUAGES

Database languages are specialized languages used to interact with a database. They allow users to perform different tasks such as defining, controlling and manipulating the data. There are several types of database languages in DBMS, categorized into the following four main types:




1. DDL (Data Definition Language)
2. DCL (Data Control Language)
3. DML (Data Manipulation Language)
4. TCL (Transaction Control Language)

# 1. DDL (Data Definition Language)

The DDL stands for Data Definition Language, Which is used to define the database's internal structure and Pattern of the Database. It is used to define and modify the structure of the database itself, including the tables, views, indexes and other schema-related objects. It deals with the creation and modification of database schema, but it doesn't deal with the data itself. Following are the five DDL commands in SQL:

- **CREATE:** Used to create database objects like tables, indexes or views.
- **ALTER:** Used to modify the structure of an existing database object, such as adding a new column to a table.
- **DROP:** Used to delete database objects.
- **TRUNCATE:** Used to remove all rows from a table, without affecting the structure.
- **RENAME:** Used to change the name of a database object.

## 1.1 CREATE Command

The CREATE is a DDL command used to create databases, tables, triggers and other database objects.

**Syntax:**

**CREATE TABLE Students (**
**column1 INT, column2 VARCHAR(50), column3 INT**
**);**

Example:

```
mysql> CREATE TABLE Students (
    ->     StudentID INT PRIMARY KEY,
    ->     Name VARCHAR(50),
    ->     Age INT
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> desc students;
+-----------+-------------+------+-----+---------+-------+
| Field     | Type        | Null | Key | Default | Extra |
+-----------+-------------+------+-----+---------+-------+
| StudentID | int         | NO   | PRI | NULL    |       |
| Name      | varchar(50) | YES  |     | NULL    |       |
| Age       | int         | YES  |     | NULL    |       |
+-----------+-------------+------+-----+---------+-------+
3 rows in set (0.01 sec)
```

## 1.2 Alter Command

ALTER is a DDL command which changes or modifies the existing structure of the database and it also changes the schema of database objects. We can also add and drop constraints of the table using the ALTER command.

**Syntax:**

**ALTER TABLE Students ADD column_name;**

Example:

```
mysql> ALTER TABLE Students
    -> ADD Weight DECIMAL(5,2);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc students;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| StudentID | int          | NO   | PRI | NULL    |       |
| Name      | varchar(50)  | YES  |     | NULL    |       |
| Age       | int          | YES  |     | NULL    |       |
| Weight    | decimal(5,2) | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

## 1.3 Drop Command

DROP is a DDL command used to delete/remove the database objects from the SQL database. We can easily remove the entire table, view or index from the database using this DDL command.

**Syntax:**

**DROP Table Table_name;**

Example:

```
mysql> drop table students;
Query OK, 0 rows affected (0.05 sec)
```

## 1.4 Truncate Command

The TRUNCATE command is used to delete all the records from a table without removing the structure. Unlike the DELETE command, which can remove specific rows and can be rolled back, TRUNCATE is a more efficient way to delete all rows in a table without logging individual row deletions.

**Syntax:**

**TRUNCATE TABLE table_name;**

Example:

```
mysql> select * from students;
+-----------+----------+------+--------+
| StudentID | Name     | Age  | Weight |
+-----------+----------+------+--------+
|         1 | John Doe |   20 |  65.50 |
+-----------+----------+------+--------+
1 row in set (0.02 sec)

mysql> TRUNCATE TABLE Students;
Query OK, 0 rows affected (0.04 sec)

mysql> select * from students;
Empty set (0.00 sec)
```

## 1.5 Rename Command

The RENAME command in a Database Management System (DBMS) is used to change the name of a database object, such as a table, column or index. This command is helpful when we want to give a more meaningful or appropriate name to an object without needing to recreate it.

**Syntax:**

**ALTER TABLE Old_Table_Name RENAME TO New_Table_Name;**

Example:

```
mysql> ALTER TABLE Students
    -> RENAME TO ClassMembers;
Query OK, 0 rows affected (0.02 sec)
```

## 2. DCL (Data Control Language)

DCL stands for Data Control Language. It is used to control the access permissions of users to the database. DCL commands help grant or revoke privileges to users, determining who can perform actions like reading or modifying data. DCL commands are transactional, meaning they can be rolled back if necessary. The two main DCL commands are:

- **Grant:** Gives user access to the database
- **Revoke:** Removes access or permissions from the user

### 2.1 Grant Command

The GRANT command in a Database Management System (DBMS) is used to provide specific permissions or privileges to users or roles. This command allows administrators to control access to database objects, ensuring that only authorized users can perform certain actions, such as selecting, inserting, updating or deleting data.

**Syntax:**

**GRANT privileges ON object TO user_or_role [WITH GRANT OPTION];**

Example:

**GRANT SELECT, INSERT ON students TO user;**

### 2.2 Revoke Command

The REVOKE command in a Database Management System (DBMS) is used to remove previously granted permissions or privileges from users or roles. This command is essential for managing access control, ensuring that users do not have more privileges than necessary to perform their tasks.

**Syntax:**

**REVOKE privileges ON object FROM user_or_role;**

Example:

**REVOKE ALL PRIVILEGES ON students FROM user;**

## 3. DML (Data Manipulation Language)

The DML (Data Manipulation Language) is used toto manage and manipulate data within a database. With DML, you can perform various operations such as inserting, updating, selecting and deleting data. These operations allow you to work with the actual content in your database tables. Here are the key DML commands:

- **SELECT:** Retrieves data from the table based on specific criteria.
- **INSERT:** Adds new rows of data into an existing table.
- **UPDATE:** Modifies existing data in a table.
- **DELETE:** Removes data from a table.
- **MERGE:** Performs an "upsert" operation, which means updating existing records or inserting new ones if they don't exist.
- **CALL:** Executes stored procedures or functions.
- **LOCK TABLE:** Prevents other users from accessing the table while changes are being made.

### 3.1 SELECT Command

The SELECT command in SQL (Structured Query Language) is used to retrieve data from one or more tables in a database. It is the most commonly used commands in SQL, allowing users to specify which columns and rows of data they want to retrieve and how they want that data organized. The SELECT statement can be used in various ways, such as selecting all data from a table, filtering records based on conditions or sorting the results.

**Syntax:**

**SELECT * FROM Table_Name**

Example:

```
mysql> select * from ClassMembers;
+-----------+----------+------+--------+
| StudentID | Name     | Age  | Weight |
+-----------+----------+------+--------+
|         1 | John Doe |   20 |  65.50 |
+-----------+----------+------+--------+
1 row in set (0.00 sec)
```

## 3.2 Insert Command

The INSERT command in SQL (Structured Query Language) is used to add new records or rows to a table in a database. It is a key operation in database management, essential for populating tables with new data. This command can insert data into all columns or specific columns of a table.

**Syntax:**

**INSERT INTO Table_Name (Column 1, Column 2, Column 3, Column 4) VALUES (Value 1, Value 2,Value 3, Value 4);**

Example:

```
mysql> INSERT INTO ClassMembers (StudentID, Name, Age, Weight)
    -> VALUES (2, 'Alice Smith', 22, 60.2);
Query OK, 1 row affected (0.03 sec)

mysql> select * from ClassMembers;
+-----------+-------------+------+--------+
| StudentID | Name        | Age  | Weight |
+-----------+-------------+------+--------+
|         1 | John Doe    |   20 |  65.50 |
|         2 | Alice Smith |   22 |  60.20 |
+-----------+-------------+------+--------+
2 rows in set (0.00 sec)
```

## 3.3 Update Command

The UPDATE command in SQL (Structured Query Language) is used to modify existing records in a table. This command enables users to change the values of one or more columns for specific rows based on a condition (criteria). It is important for maintaining and adjusting data in a database when necessary.

**Syntax:**

**UPDATE Table_Name SET Name = 'New_Value' WHERE Name = 'Ola_Value';**

Example:

```
mysql> UPDATE ClassMembers
    -> SET Name = 'Roman'
    -> WHERE Name = 'John Doe';
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from ClassMembers;
+-----------+-------------+------+--------+
| StudentID | Name        | Age  | Weight |
+-----------+-------------+------+--------+
|         1 | Roman       |   20 |  65.50 |
|         2 | Alice Smith |   22 |  60.20 |
+-----------+-------------+------+--------+
2 rows in set (0.00 sec)
```

## 3.4 Merge Command

The MERGE command is used to perform **INSERT**, **UPDATE**, or **DELETE** operations in a single statement depending on whether a condition matches or not.

It is especially useful when **synchronizing two tables** — for example, updating a target table based on changes in a source table.

**Syntax of MERGE:**

```
MERGE INTO target_table USING source_table
ON (condition)
WHEN MATCHED THEN
    UPDATE SET column1 = value1, column2 = value2, ...
WHEN NOT MATCHED THEN
    INSERT (column1, column2, ...) VALUES (value1, value2, ...);
```

**Example:**

Assume we have two tables:

1. **employees** (target table):

```sql
employee_id | name     | department
------------|----------|------------
101         | John     | HR
102         | Alice    | Sales
```

2. **new_employees** (source table):

```sql
employee_id | name     | department
------------|----------|------------
101         | John     | Finance     -- Department updated
103         | Bob      | Marketing   -- New employee
```

MERGE Command:

```
MERGE INTO employees AS e
USING new_employees AS ne
ON (e.employee_id = ne.employee_id)
WHEN MATCHED THEN
    UPDATE SET e.department = ne.department
WHEN NOT MATCHED THEN
    INSERT (employee_id, name, department)
    VALUES (ne.employee_id, ne.name, ne.department);
```

Result in employees table after MERGE:

```
sql

employee_id | name     | department
------------|----------|------------
101         | John     | Finance      -- Updated
102         | Alice    | Sales
103         | Bob      | Marketing    -- Inserted
```

## 3.5 Delete Command

The DELETE command in SQL (Structured Query Language) is used to remove one or more existing records from a table in a database. It is an essential operation for managing data, enabling users to delete specific rows that meet certain conditions or criteria.

**Syntax:**

DELETE FROM Table_Name WHERE Column = Value;

Example:

```
mysql> DELETE FROM ClassMembers
    -> WHERE StudentID = 2;
Query OK, 1 row affected (0.03 sec)

mysql> select * from ClassMembers;
+-----------+-------+------+--------+
| StudentID | Name  | Age  | Weight |
+-----------+-------+------+--------+
|         1 | Roman |   20 |  65.50 |
+-----------+-------+------+--------+
1 row in set (0.00 sec)
```

## 3.6 CALL Command

The Call command is used to invoke a stored procedure or user-defined function, which is a set of precompiled SQL statements. It allows you to execute complex operations within a database as a single unit.

**Syntax:**

CALL user_defined_function(parameter 1, parameter 2);

**Example:**

CALL UpdateEmployeeSalary(101, 55000);

### 3.7 LOCK TABLE

The lock table command is used to lock the table for preventing access from others, ensuring no other operations (like insertions, updates or deletions) can be performed on the table while it's locked. Useful for transactional operations where consistency is important.

**Syntax:**

LOCK TABLE your_table IN EXCLUSIVE MODE;

**Example:**

LOCK TABLE ClassMembers IN EXCLUSIVE MODE;

## 4. TCL ( Transaction Control Language )

The TCL full form is Transaction Control Language commands are used to manage and control transactions in a database, grouping them into logical units. These commands help ensure the integrity of data and consistency during complex operations. Here are the two main commands in this category:

- **Commit:** Saves all the changes made during the current transaction to the database. These are very useful in the banking sector.
- **Rollback:** used to restore the database to its original state from the last commit. This command also plays an important role in Banking Sectors.

### 4.1 Commit Command

The COMMIT command is used to save all changes made during a transaction in the database. This command ensures that modifications made by DML statements (such as INSERT, UPDATE or DELETE) become permanent in the database.

**Syntax:**

Database Operation

Commit

**Example:**

```
mysql> INSERT INTO ClassMembers (StudentID, Name, Age, Weight)
    -> VALUES (4, 'Emma Johnson', 23, 50);
Query OK, 1 row affected (0.03 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

## 4.2 ROLLBACK Command

The rollback command is used to restore the database to its state at the last COMMIT, effectively undoing any changes made since that point. It helps ensure data consistency by allowing the reversal of partial or erroneous operations.

**Syntax:**

ROLLBACK;

**Example:**

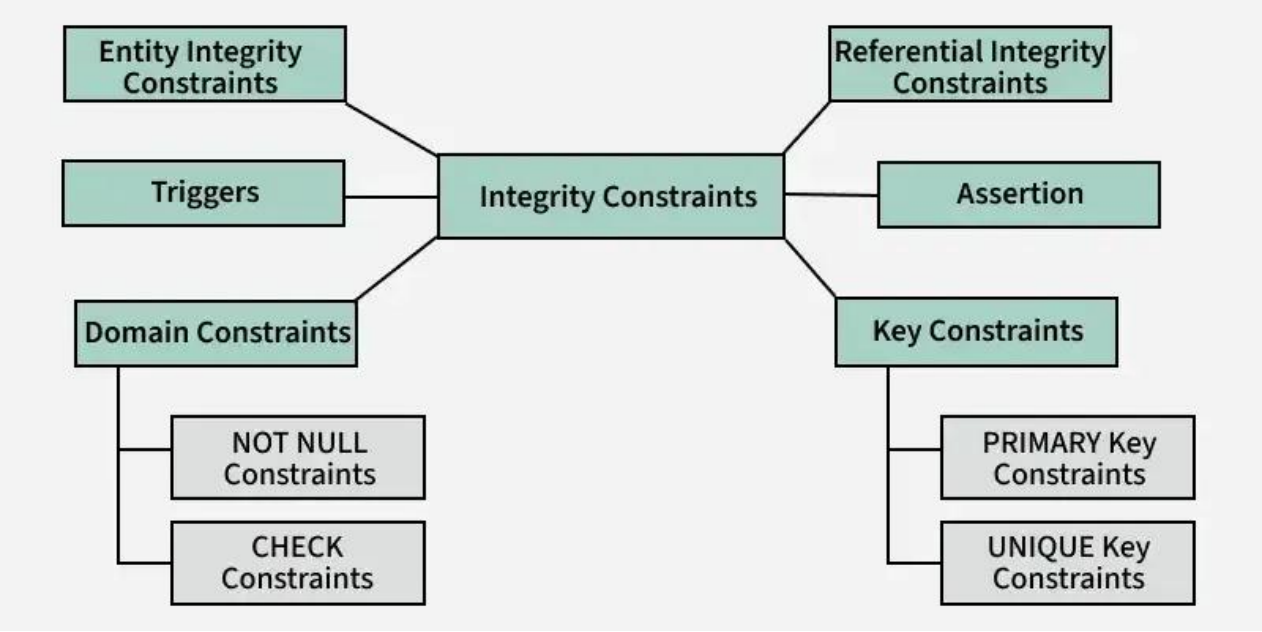
```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> commit;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM ClassMembers where StudentID = 4;
Query OK, 1 row affected (0.03 sec)

mysql> rollback;
Query OK, 0 rows affected (0.00 sec)
```

## Integrity Constraints

Integrity constraints are a set of rules used in DBMS to ensure that the data in a database is accurate, consistent and reliable. These rules helps in maintaining the quality of data by ensuring that the processes like adding, updating or deleting information do not harm the integrity of the database. Integrity constraints also define how different parts of the database are connected and ensure that these relationships remain valid. They play an essential role in making sure the data is meaningful and follows the logical structure of the database.

**What are Integrity Constraints ?**

Integrity constraints in a Database Management System are rules that help keep the data in a database accurate, consistent and reliable. They act like a set of guidelines that ensure all the information stored in the database follows specific standards.




**Types of Integrity Constraints**

There are Different types of Integrity Constraints used in DBMS, these are:

1. Domain Constraints
2. Entity Integrity Constraints
3. Key Constraints
4. Referential integrity constraints

5. Assertion
6. Triggers

**1. Domain Constraints**

Domain constraints are a type of integrity constraint that ensure the values stored in a column (or attribute) of a database are valid and within a specific range or domain. In simple terms, they define what type of data is allowed in a column and restrict invalid data entry. The data type of domain include string, char, time, integer, date, currency etc. The value of the attribute must be available in comparable domains.

**Example:** Below table demonstrates domain constraints in action by enforcing rules for each column

| Student_Id | Name | Semester | Age |
|---|---|---|---|
| 21CSE100 | Aniket Kumar | 6th | 20 |
| 21CSE101 | Shashwat Dubey | 7th | 21 |
| 21CSE102 | Manvendra Sharma | 8th | 22 |
| 21CSE103 | Ashmit Dubey | 5th | 20 |

1. **Student_Id:** Must be unique and follow a specific format like 21CSE###. No duplicates or invalid formats allowed.
2. **Name:** Accepts only valid text (no numbers) and cannot be left empty (NOT NULL constraint).
3. **Semester:** Allows specific values like 5th, 6th, etc., and ensures valid input (e.g., no 10th if not permitted).
4. **Age:** Must be an integer within a reasonable range (e.g., 18-30) and cannot contain invalid data like negative numbers or text.

**Types of Domain Constraints:**

- **NOT NULL Constraint:** Ensures No records can have NULL value.
- **CHECK Constraint:** This Constraint Checks for any specified condition over any attribute.

**Why Domain Constraints Are Important :**

- They prevent invalid or inconsistent data from entering the database.
- They ensure the database is reliable and follows predefined business rules.
- They make the database easier to manage and maintain by reducing errors.

**2. Entity Integrity Constraints**

Entity integrity constraints state that primary key can never contain null value because primary key is used to determine individual rows in a relation uniquely, if primary key contains null value then we cannot identify those rows. A table can contain null value in it except primary key field.

**Key Features of Entity Integrity Constraints:**

- **Uniqueness:** The primary key value must be unique for each row in the table. No duplicate entries are allowed in the primary key column.
- **NOT NULL:** The primary key column cannot contain NULL values, as every row must have a valid identifier.
- **Essential for Table Design:** Ensures that every record in the table can be uniquely identified, preventing ambiguity.

**Example: I**t is not allowed because it is containing primary key (Student_id) as NULL value.

| Student_id | Name | Semester | Age |
|---|---|---|---|
| 21CSE101 | Ramesh | 5th | 20 |
| 21CSE102 | Kamlesh | 5th | 21 |
| 21CSE103 | Aakash | 5th | 22 |
| NULL | Mukesh | 5th | 20 |

**3. Key Constraints**

Key constraints ensure that certain columns or combinations of columns in a table uniquely identify each row. These rules are essential for maintaining data integrity and preventing duplicate or ambiguous records.

**Why Key Constraints Are Important ?**

- Prevent Duplicates: Ensure unique identification of rows.
- Maintain Relationships: Enable proper linking between tables (via foreign keys).
- Enforce Data Integrity: Prevent invalid or inconsistent data.

**Example:** It is now acceptable because all rows must be unique.

| Student_id | Name | Semester | Age |
|---|---|---|---|
| 21CSE101 | Ramesh | 5th | 20 |
| 21CSE102 | Kamlesh | 5th | 21 |
| 21CSE103 | Aakash | 5th | 22 |
| 21CSE102 | Mukesh | 5th | 20 |

**4. Referential integrity constraints**

Referential integrity constraints are rules that ensure relationships between tables remain consistent. They enforce that a foreign key in one table must either match a value in the referenced primary key of another table or be NULL. This guarantees the logical connection between related tables in a relational database.

**Why Referential Integrity Constraints Are Important ?**

- Maintains Consistency: Ensures relationships between tables are valid.
- Prevents Orphan Records: Avoids cases where a record in a child table references a non-existent parent record.
- Enforces Logical Relationships: Strengthens the logical structure of a relational database.

**Example:** Here, in below example Block_No 22 entry is not allowed because it is not present in 2nd table.

| Student_id | Name | Semester | Block_No |
|---|---|---|---|
| 22CSE101 | Ramesh | 5th | 20 |
| 21CSE105 | Kamlesh | 6th | 21 |
| 22CSE102 | Aakash | 5th | 20 |
| 23CSE106 | Mukesh | 2nd | 22 |

| Block_No | Block Location |
|---|---|
| 20 | Chandigarh |
| 21 | Punjab |
| 25 | Delhi |

**5. Assertion**

An assertion is a declarative mechanism in a database that ensures a specific condition or rule is always satisfied across the entire database. It is a global integrity constraint, meaning it applies to multiple tables or the entire database rather than being limited to a single table or column. An assertion in SQL-92 takes the form:

*create assertion <assertion-name> check <predicate>*

When an assertion is made, the system tests it for validity. This testing may introduce a significant amount of overhead; hence assertions should be used with great care.

**Example of an Assertion:**

```
CREATE ASSERTION sum_constraint
CHECK (
NOT EXISTS (
SELECT *
FROM branch
WHERE (
SELECT SUM(amount)
FROM loan
WHERE loan.branch_name = branch.branch_name
) >= (

SELECT SUM(amount)
FROM account
WHERE account.branch_name = branch.branch_name
)
)
);
```

**Explanation:**

The following SQL statement creates an assertion to ensure that the total loan amount at each branch is always less than the total account balances at the same branch.

- **Purpose:** This assertion enforces a global business rule. The sum of all loan amounts for a branch must always be less than the sum of all account balances in the same branch. This prevents branches from issuing loans beyond their financial capacity.
- **Subqueries:** The first subquery (SELECT SUM(amount) FROM loan) calculates the total loan amount for each branch. The second subquery (SELECT SUM(amount) FROM account) calculates the total balance of accounts for the same branch.
- **Condition:** The NOT EXISTS clause ensures there is no branch where the loan amount is greater than or equal to the account balance.
- **Behavior:** If a transaction (e.g., inserting a loan or updating an account) violates this rule, the operation will be rejected by the database.

**6. Triggers**

A trigger is a procedural statement in a database that is automatically executed in response to certain events such as INSERT, UPDATE, or DELETE. Triggers are often used to enforce complex integrity constraints or implement business rules that cannot be captured using standard constraints like primary keys or foreign keys.

**Example SQL Trigger:**

```
CREATE TRIGGER handle_overdraft
AFTER UPDATE ON account
FOR EACH ROW
BEGIN
-- Check if the balance has become negative after the update
IF NEW.balance < 0 THEN
-- Set the account balance to zero
UPDATE account
SET balance = 0
WHERE account_number = NEW.account_number;

-- Create a loan record with the same account number as the loan number
INSERT INTO loan (loan_number, loan_amount)
VALUES (NEW.account_number, ABS(NEW.balance)); -- ABS to ensure positive loan amount
```

```
END IF;
END;
```

**Explanation:**

- **Trigger Type:** The trigger runs after an update on the account table.
- **Condition:** It checks if the account balance is negative after the update.
- **Actions:** If the balance is negative, the account balance is reset to zero. A loan is created with the same account number as the loan number, and the loan amount is the absolute value of the negative balance. The ABS() function ensures the loan amount is positive.

## MySQL/PostgreSQL: overview; features; datatypes; Standardization guidelines

**Overview**

**MySQL**

- **Type:** Open-source Relational Database Management System (RDBMS)
- **Developer:** Originally MySQL AB → now owned by Oracle Corporation
- **License:** GPL (Community Edition) and Commercial versions available
- **Use Cases:** Web applications, CMS (WordPress, Joomla), small–medium scale enterprise apps
- **Strength:** Simplicity, high speed in read-heavy workloads, large community support

**PostgreSQL**

- **Type:** Open-source Object-Relational Database Management System (ORDBMS)
- **Developer:** PostgreSQL Global Development Group
- **License:** PostgreSQL License (permissive, similar to MIT)
- **Use Cases:** Applications requiring advanced SQL features, high data integrity, geospatial analysis (PostGIS)
- **Strength:** Standards compliance, extensibility, ACID compliance, complex queries

**Features**

| Feature | MySQL | PostgreSQL |
|---|---|---|
| ACID Compliance | Yes (with InnoDB engine) | Yes (always) |
| SQL Standards Support | Partial | Very High (SQL:2011) |
| Performance | Fast for read-heavy workloads | Better for complex queries and write-heavy workloads |
| Extensibility | Limited | Highly extensible (custom types, operators) |
| Replication | Supported (Master–Slave, Group Replication) | Supported (Streaming Replication, Logical Replication) |
| Indexing | B-Tree, Full-text, Spatial (with plugins) | B-Tree, Hash, GiST, GIN, SP-GiST, BRIN |
| JSON Support | Yes (basic functions) | Advanced JSONB support |
| Geospatial Support | Limited (via plugins) | Excellent (PostGIS) |
| Procedural Languages | Stored procedures in SQL/JavaScript | PL/pgSQL, PL/Python, PL/Perl, etc. |

**Datatypes**

**MySQL Datatypes**

- **Numeric:** INT, BIGINT, FLOAT, DOUBLE, DECIMAL
- **Date/Time:** DATE, DATETIME, TIMESTAMP, TIME, YEAR
- **String:** CHAR, VARCHAR, TEXT (TINYTEXT, TEXT, MEDIUMTEXT, LONGTEXT)
- **Binary:** BLOB (TINYBLOB, BLOB, MEDIUMBLOB, LONGBLOB)
- **JSON:** JSON (from version 5.7)

**PostgreSQL Datatypes**

- **Numeric:** SMALLINT, INTEGER, BIGINT, NUMERIC, REAL, DOUBLE PRECISION
- **Date/Time:** DATE, TIME, TIMESTAMP, TIMESTAMPTZ, INTERVAL
- **Character:** CHAR, VARCHAR, TEXT
- **Binary:** BYTEA

- **Boolean:** BOOLEAN
- **JSON:** JSON, JSONB (binary optimized)
- **Special:** ARRAY, UUID, ENUM, CIDR (IP addresses), GEOMETRY (via PostGIS)

**Standardization Guidelines**

**W**hen designing databases in MySQL or PostgreSQL, follow these best practices:

**a. SQL Standard Compliance**

- Write ANSI SQL-compliant queries to ensure portability between MySQL and PostgreSQL.
- Avoid using vendor-specific syntax unless absolutely required.

**b. Naming Conventions**

- Use snake_case for table and column names (employee_details instead of EmployeeDetails).
- Avoid reserved keywords (user, order).
- Be descriptive but concise (customer_id instead of id1).

**c. Data Normalization**

- Apply normal forms (1NF, 2NF, 3NF) to reduce redundancy.
- Denormalize only when there's a performance need.

**d. Primary & Foreign Keys**

- Always define primary keys.
- Use foreign keys with ON DELETE / ON UPDATE rules for data integrity.

**e. Data Types Selection**

- Choose the smallest suitable data type (e.g., SMALLINT instead of INT when possible).
- Use TEXT only when string length is unpredictable; prefer VARCHAR for fixed limits.

**f. Indexing**

- Create indexes on frequently queried columns.
- Avoid over-indexing (it slows down writes).

**g. Security**

- Use roles and permissions to control access.
- Avoid using the root/superuser account for applications.